FUNCTION DEFINITION COMPILER

A PROJECT REPORT

Group 7
submitted by

RUTH MARIAM JOSEPH
AJC24CS183

RYAN JACOB
AJC24CS184

TEESA THOMAS
AJC24CS205

THOMAS PETER
AJC24CS208

YASH BIJO
AJC24CS212

ZEN KOSHY
AJC24CS213

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirement for the award of the Degree

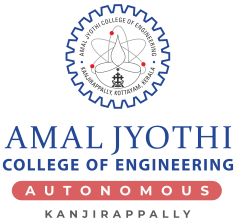
of

Bachelor of Technology

in

Computer Science and Engineering

Under the guidance of
Ms. TINTU ALPHONSA THOMAS
Assistant Professor



Department of Computer Science and Engineering
Amal Jyothi College of Engineering(Autonomous)
Kanjirappally-686518

October 2026

FUNCTION DEFINITION COMPILER

PROJECT REPORT

Group Number:	Group 7
Project Title:	Function Definition Compiler
Team Members:	1. Ruth Mariam Joseph (Roll no: 39) 2. Ryan Jacob (Roll no: 40) 3. Teesa Thomas (Roll no: 61) 4. Thomas Peter (Roll no: 64) 5. Yash Bijo (Roll no: 68) 6. Zen Koshy (Roll no: 69
Guide:	Tintu Alphonsa Thomas Assistant Professor Dept. of CSE
Submission Date:	October 23rd 2026

TABLE OF CONTENTS

1. Abstract.....	3
2. Introduction & Problem Statement.....	3
3. Objectives.....	4
4. Existing System and Proposed System.....	4
5. System Design & Architecture.....	5
6. Context-Free Grammar and Grammar Description.....	5
7. Lexical Analyzer.....	7
8. Syntax Analyzer & Parser.....	7
9. Abstract Syntax Tree (AST) & Semantic Analysis.....	7
10. Testing & Test Cases.....	8
11. Source Code Files & Git Repository Structure.....	11
12. Execution Screenshots & Validation Results.....	12
13. Individual Contribution Details.....	16
14. Conclusion.....	17

1. Abstract

The Function Definition Compiler is a compiler-design mini-project that demonstrates the front end of a simplified programming language focused on function definitions and related statements. The project is implemented in Python and includes a lexical analyzer, recursive-descent parser, Abstract Syntax Tree (AST) node structures, semantic analyzer, and a user interface. The lexical analyzer identifies keywords, identifiers, numbers, operators, and punctuation. The parser checks whether the input follows the defined Context-Free Grammar (CFG), which supports function definitions, parameter lists, assignments, function calls, print statements, return statements, and arithmetic expressions. The AST represents the meaningful structure of the parsed program, while semantic analysis performs the checks implemented in the project. The interface allows users to enter source code and inspect compiler results. This is an educational compiler front end: it validates and represents source programs and does not, by itself, imply execution or machine-code generation.

2. Introduction & Problem Statement

2.1 Introduction

A compiler is system software that analyzes a source program and may translate it into another form, such as intermediate code or machine code. Compiler construction is commonly divided into stages, including lexical analysis, syntax analysis, semantic analysis, intermediate representation, optimization, and code generation. This project focuses on the front-end stages needed to recognize function definitions and related constructs. The Python implementation separates token recognition, parsing, AST representation, and semantic checks into different modules. The grammar includes optional parameter and argument lists, statement sequences, function calls, return statements, print statements, assignments, and arithmetic expressions with operator precedence.

2.2 Problem Statement

Beginners may find it difficult to understand how a compiler recognizes function declarations, parses parameters and arguments, groups expressions according to precedence, and reports invalid source code. The aim of this project is to develop a small compiler front end that recognizes the constructs described by the project's CFG, builds an AST for valid syntax, and performs supported semantic checks. The system should identify malformed input, such as missing delimiters or invalid expressions, and present compiler results in an understandable form. The implementation is

limited to the grammar and checks provided by the project and is not a complete general-purpose compiler.

3. Objectives

- To define a Context-Free Grammar for function definitions, parameter lists, statements, calls, and expressions.
- To implement a lexical analyzer that recognizes keywords, identifiers, numbers, operators, and punctuation.
- To implement a parser that validates source programs against the CFG.
- To construct an Abstract Syntax Tree for syntactically valid programs.
- To perform supported semantic checks on function declarations, parameters, and calls.
- To report lexical, syntax, and semantic errors where implemented.
- To test the compiler with valid and invalid input programs.
- To provide a user interface for entering source code and viewing compiler outputs.

4. Existing System and Proposed System

4.1 Existing System

A full compiler supports a broad programming language and may include complex stages such as type checking, optimization, intermediate-code generation, and target-code generation. This can make the relationship between individual compiler stages difficult for beginners to observe. A focused educational implementation is useful for demonstrating how source text becomes tokens, how tokens are validated against a grammar, and how a syntax tree represents the program.

4.2 Proposed System

The proposed system is a Python-based compiler front end organized into lexical analysis, parsing, AST node representation, and semantic analysis. The grammar supports function definitions using def, begin, and end; parameter and argument lists; assignments; function calls; print and return statements; and arithmetic expressions using +, -, *, and /. The existing repository also contains app.py, main.py, templates/, and requirements.txt, which support the application workflow according to their implementation. The system validates input and displays available results but should not be described as executing source programs or generating machine code unless those features are separately implemented.

5. System Design & Architecture

5.1 Architecture of the System

The intended processing flow begins when a user enters source code in the interface. The application passes the source text to the lexical analyzer, which produces tokens. The parser consumes those tokens and checks the input against the grammar. During or after parsing, AST node structures represent the program's syntax. The semantic analyzer performs the semantic checks supported by the current implementation. The application then presents the relevant status, tokens, syntax-tree representation, and errors. The exact route and function names depend on the current code in app.py and main.py.

5.2 Modules of the System

Module	Description
Lexical Analysis (lexer.py)	Scans source text and recognizes supported keywords, identifiers, numbers, operators, and punctuation.
Syntax Analysis (parser.py)	Checks whether the token sequence conforms to the grammar and identifies syntax errors.
AST Node Structures (ast_nodes.py)	Defines node structures used to represent program constructs and expressions in tree form.
Semantic Analysis (semantic_analyzer.py)	Performs semantic validation rules implemented by the project.
Application Interface (app.py, main.py, templates/)	Provides the entry point and user-facing workflow according to the current application code.
Grammar Specification (grammar.txt)	Documents the productions used to define the accepted language.

6. Context-Free Grammar and Grammar Description

6.1 Context-Free Grammar (CFG)

A Context-Free Grammar (CFG) describes the syntax of a language using terminals, non-terminals, production rules, and a start symbol. The grammar below is the grammar supplied for this Function Definition Compiler.

```
Program     → TopList
TopList     → Statement ; TopList | ε
Statement   → FunctionDef | Assignment | FunctionCall | PrintStmt
```

```
FunctionDef  → def id ( Params ) begin Body end
Params       → id ParamTail | ε
ParamTail    → , id ParamTail | ε
Body         → BodyStmt ; Body | ε
BodyStmt     → Assignment | FunctionCall | ReturnStmt | PrintStmt

ReturnStmt   → return Expr
Assignment   → id = Expr
PrintStmt    → print ( Expr )
FunctionCall → id ( Args )
Args         → Expr ArgTail | ε
ArgTail      → , Expr ArgTail | ε

Expr         → Term ExprTail
ExprTail     → + Term ExprTail | - Term ExprTail | ε
Term         → Factor TermTail
TermTail     → * Factor TermTail | / Factor TermTail | ε
Factor       → id | number | FunctionCall | ( Expr )
```

6.2 Terminals and Start Symbol

Terminals:

```
def, begin, end, return, print, id, number, (, ), ,, ;, =, +, -, *, /
```

Start symbol:

```
Program
```

6.3 Grammar Notes

- ε represents an empty production.
- Params permits either a comma-separated list of identifiers or no parameters.
- Args permits either a comma-separated list of expressions or no arguments.
- Expr and Term separate addition/subtraction from multiplication/division, giving multiplication and division higher precedence.
- Each statement in TopList and Body is followed by a semicolon according to the supplied grammar.
- The grammar describes syntax; semantic rules such as duplicate function names or argument-count matching must be implemented separately.

7. Lexical Analyzer

Lexical analysis is the first stage of the compiler front end. The lexical analyzer in lexer.py is responsible for recognizing the token categories supported by the implementation. Based on the supplied grammar, these include the keywords def, begin, end, return, and print; identifiers represented by id; numeric literals represented by number; and punctuation and operators such as parentheses, commas, semicolons, assignment, addition, subtraction, multiplication, and division. The lexer passes recognized tokens to the parser. Invalid characters or unsupported token patterns should be reported as lexical errors if error handling is implemented.

8. Syntax Analyzer & Parser

The syntax analyzer is implemented in parser.py. It checks whether the sequence of tokens follows the CFG. The grammar includes function definitions, optional parameter lists, function bodies, assignments, function calls, return statements, print statements, and arithmetic expressions. The expression grammar uses separate Expr, Term, and Factor levels to represent operator precedence. The parser should reject malformed input such as missing parentheses, missing semicolons, incomplete expressions, and incorrectly formed function headers. The actual error messages depend on the current parser implementation.

9. Abstract Syntax Tree (AST) & Semantic Analysis

9.1 Abstract Syntax Tree (AST)

An Abstract Syntax Tree is a tree representation of the meaningful structure of a program. The repository contains ast_nodes.py, which defines AST node structures. The parser and AST logic should represent the program's function definitions, parameters, body statements, assignments, calls, return and print statements, and arithmetic expressions. For an expression such as a + b * 2, the tree should place multiplication below addition so that the structure reflects operator precedence, provided the AST builder follows the grammar.

9.2 Semantic Analysis

Semantic analysis checks meaning-related rules that go beyond grammatical correctness. The repository contains semantic_analyzer.py. Depending on the implemented rules, this module may check function declarations, duplicate definitions or parameters, whether called functions exist, and whether the number of arguments matches the declared parameters. Only checks present in the

code should be reported as completed features. Syntax acceptance alone does not guarantee semantic validity.

10. Testing & Test Cases

Testing should verify that supported programs are accepted and malformed programs are rejected. The following examples are based on the supplied CFG. Expected results are intended outcomes; run them through the current application and capture actual outputs before final submission.

10.1 Test Case 1: Simple function definition

Input:

```
def greet() begin
    print(10);
end;
```

Expected: lexical analysis and syntax analysis succeed; an AST is produced if AST output is integrated.

10.2 Test Case 2: Function with parameters and return

Input:

```
def add(a, b) begin
    return a + b;
end;
```

Expected: function definition, parameters, and return expression are accepted.

10.3 Test Case 3: Arithmetic precedence

Input:

```
def calculate(x, y) begin
    result = x + y * 2;
    return result;
end;
```

Expected: expression is parsed with multiplication taking precedence over addition.

10.4 Test Case 4: Function call

Input:

```
def square(x) begin
    return x * x;
end;
print(square(5));
```

Expected: function definition and call syntax are accepted, subject to the parser's supported call handling.

10.5 Test Case 5: Multiple parameters and arguments

Input:

```
def multiply(a, b, c) begin
    return a * b * c;
end;
print(multiply(2, 3, 4));
```

Expected: comma-separated parameter and argument lists are accepted.

10.6 Test Case 6: Parenthesized expression

Input:

```
def compute(a, b) begin
    result = (a + b) * 2;
    return result;
end;
```

Expected: parenthesized expression is parsed successfully.

10.7 Test Case 7: Missing closing parenthesis

Input:

```
def add(a, b begin
    return a + b;
end;
```

Expected: syntax error for a missing closing parenthesis.

10.8 Test Case 8: Missing end keyword

Input:

```
def greet() begin
    print(10);
```

Expected: syntax error for an incomplete function definition.

10.9 Test Case 9: Missing semicolon

Input:

```
def add(a, b) begin
    return a + b
end;
```

Expected: syntax error for a missing statement terminator.

10.10 Test Case 10: Invalid expression

Input:

```
def calculate(a) begin
    return a + ;
end;
```

Expected: syntax error for an incomplete arithmetic expression.

10.11 Test Case 11: Invalid assignment

Input:

```
def calculate(a) begin
    a = = 10;
end;
```

Expected: syntax error for an invalid assignment.

10.12 Semantic Test Cases

Undefined function

```
print(unknown(5));
```

Expected: semantic error only if undefined-function checking is implemented.

Duplicate parameter

```
def add(a, a) begin
    return a + a;
end;
```

Expected: semantic error only if duplicate parameter checking is implemented.

Incorrect argument count

```
def add(a, b) begin
    return a + b;
end;
print(add(5));
```

Expected: semantic error only if argument-count checking is implemented.

Duplicate function definition

```
def greet() begin
    print(1);
end;
def greet() begin
    print(2);
end;
```

Expected: semantic error only if duplicate function checking is implemented.

10.13 Testing Procedure

1. Enter the source program in the application interface or run it through the project's main entry point.
2. Inspect the token output from the lexical analyzer.
3. Check whether the parser accepts or rejects the program.
4. Inspect the AST for valid programs if AST output is available.
5. Check semantic validation messages where semantic analysis is integrated.
6. Compare the actual output with the expected behavior and record the results.
7. Capture screenshots of successful compilation and representative syntax and semantic errors for the report.

11. Source Code Files & Git Repository Structure

The following repository structure is based on the screenshot supplied for Group 7. File names are preserved as shown in the current GitHub repository.

```
function-definition-compiler/
│
├── templates/
├── README.md
├── app.py
├── ast_nodes.py
├── grammar.txt
├── lexer.py
├── main.py
├── parser.py
├── requirements.txt
└── semantic_analyzer.py
```

File / Folder	Purpose / Description
templates/	Contains the user-interface template files.
README.md	Project overview, features, installation, and run instructions.
app.py	Web application or application integration entry point, depending on implementation.
ast_nodes.py	AST node definitions used to represent parsed constructs.
grammar.txt	Context-Free Grammar for the function definition language.
lexer.py	Tokenizes source code into recognized token categories.
main.py	Main entry point for running the compiler, according to project setup.
parser.py	Syntax analyzer that checks the token sequence against the grammar.
requirements.txt	Python dependencies needed by the project.
semantic_analyzer.py	Semantic validation rules implemented for the language.

12. Execution Screenshots & Validation Results

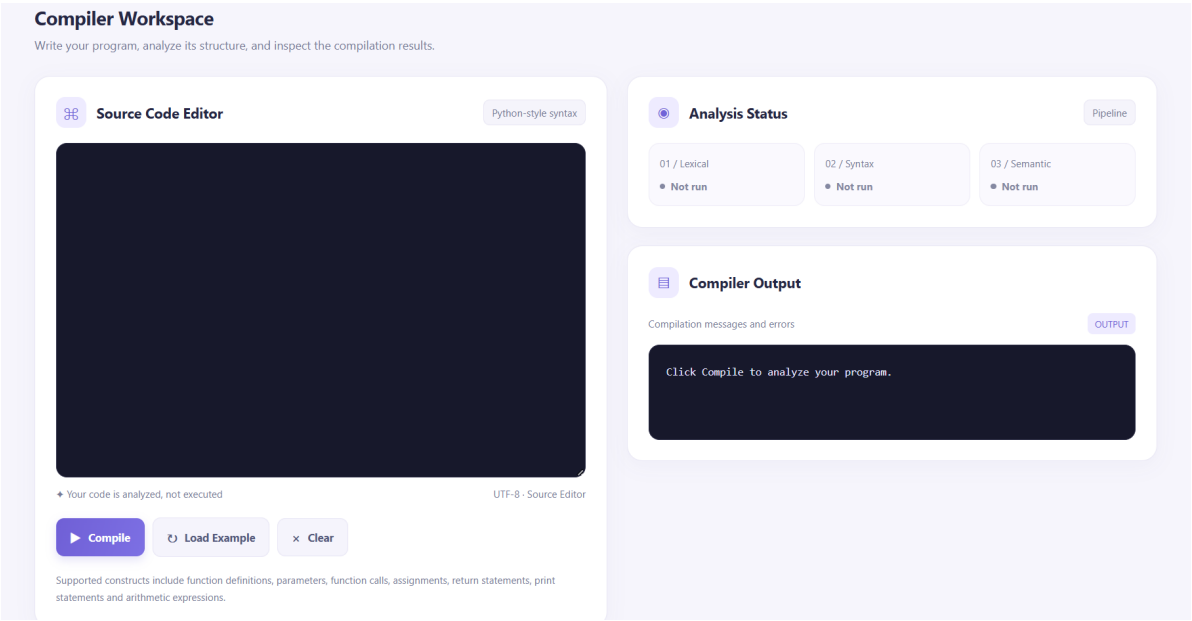


Figure 1: Compiler Interface [source-code editor and available controls]

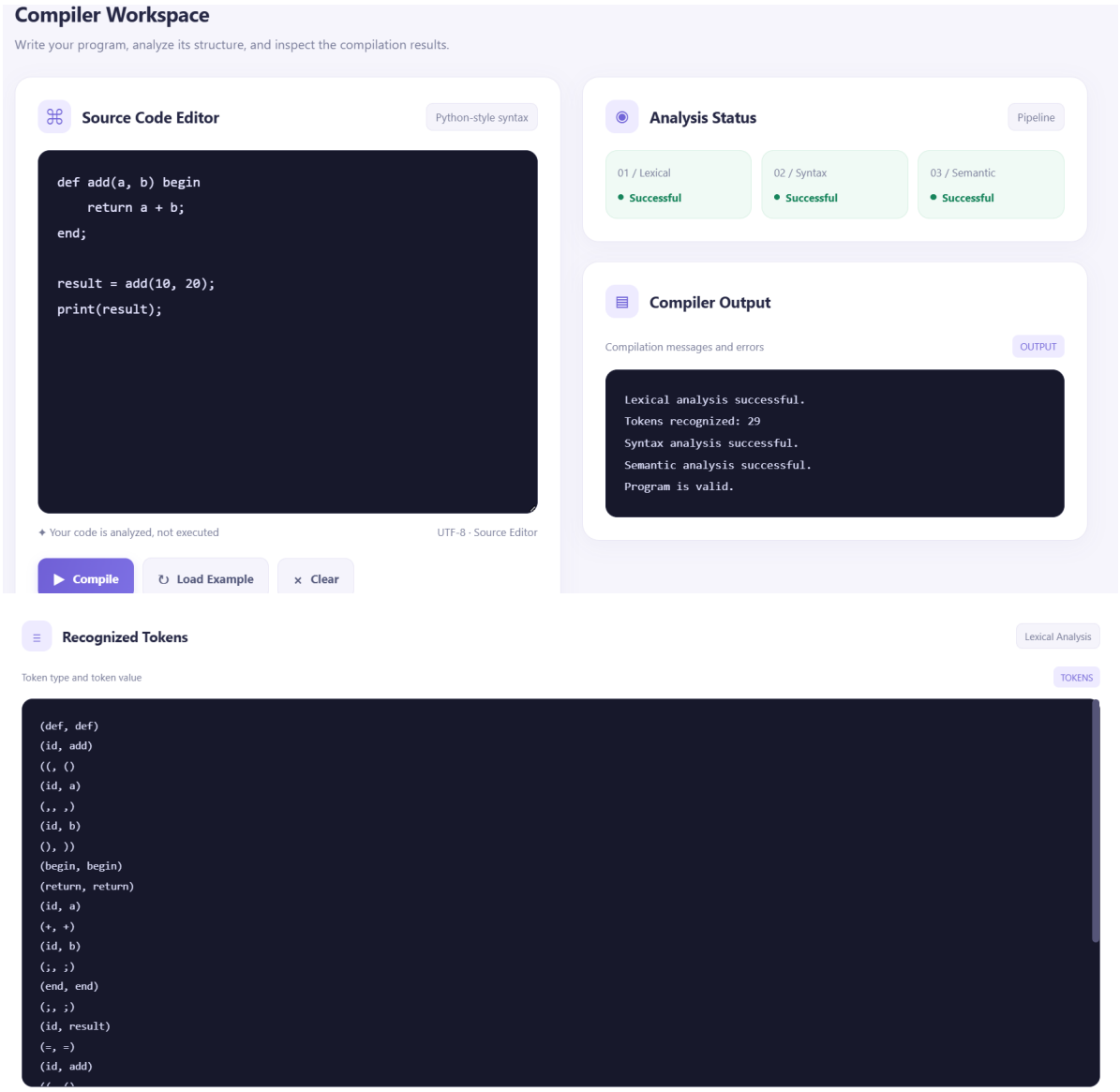


Figure 2: Successful Compilation

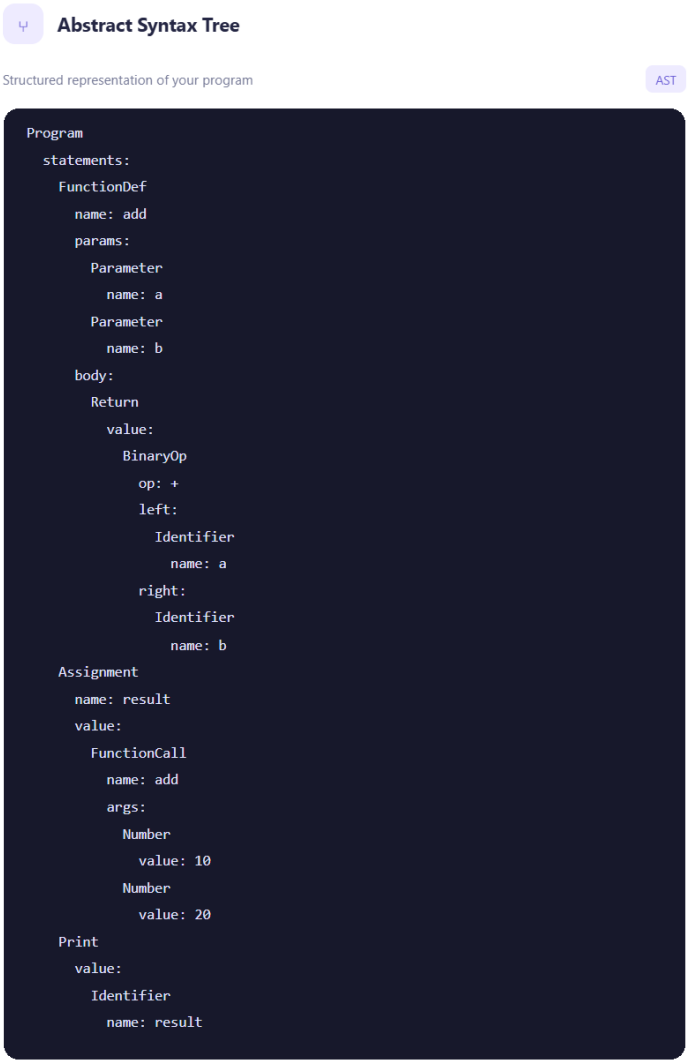


Figure 3: Abstract Syntax Tree

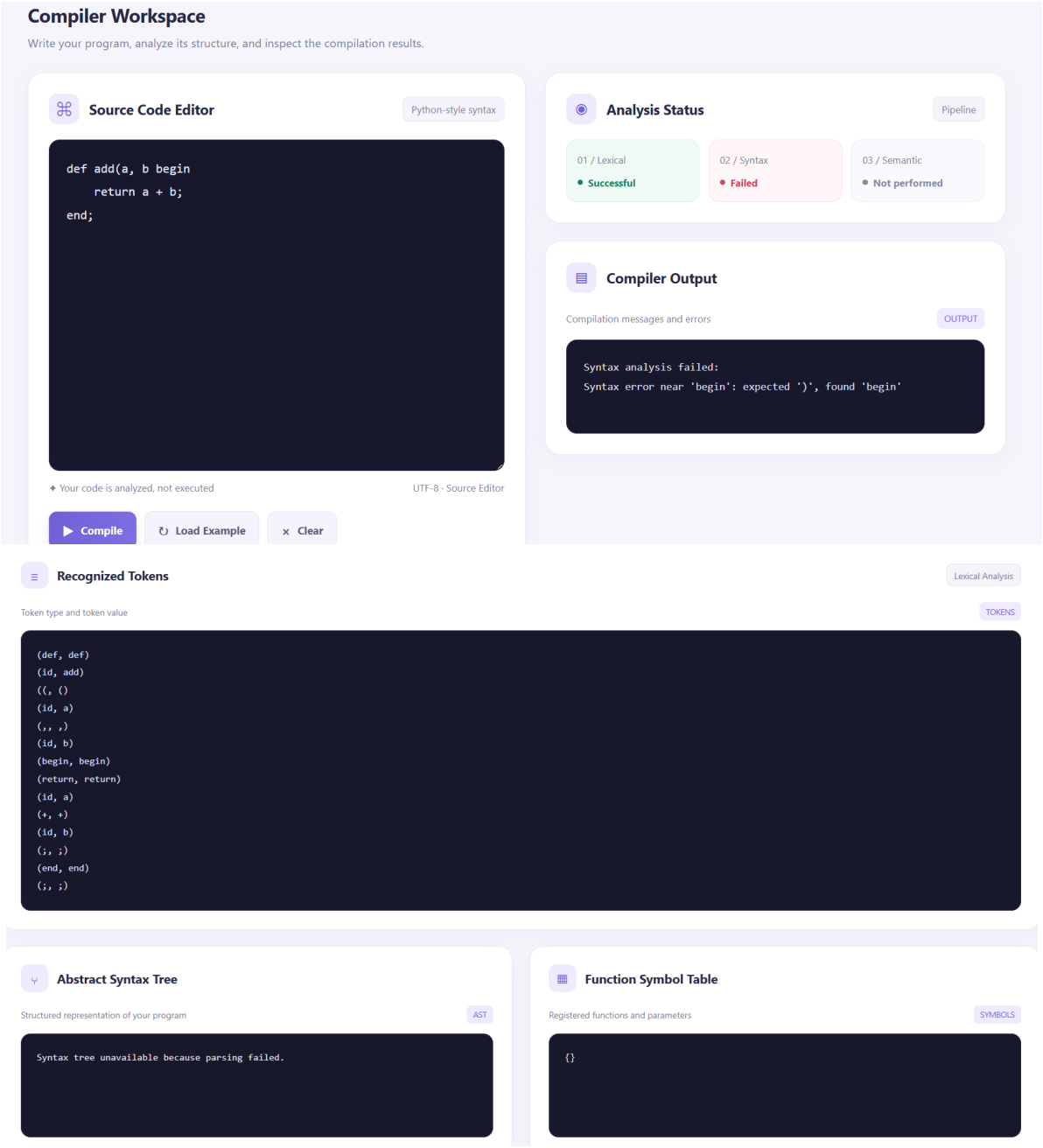


Figure 4: Syntax Analysis Failure

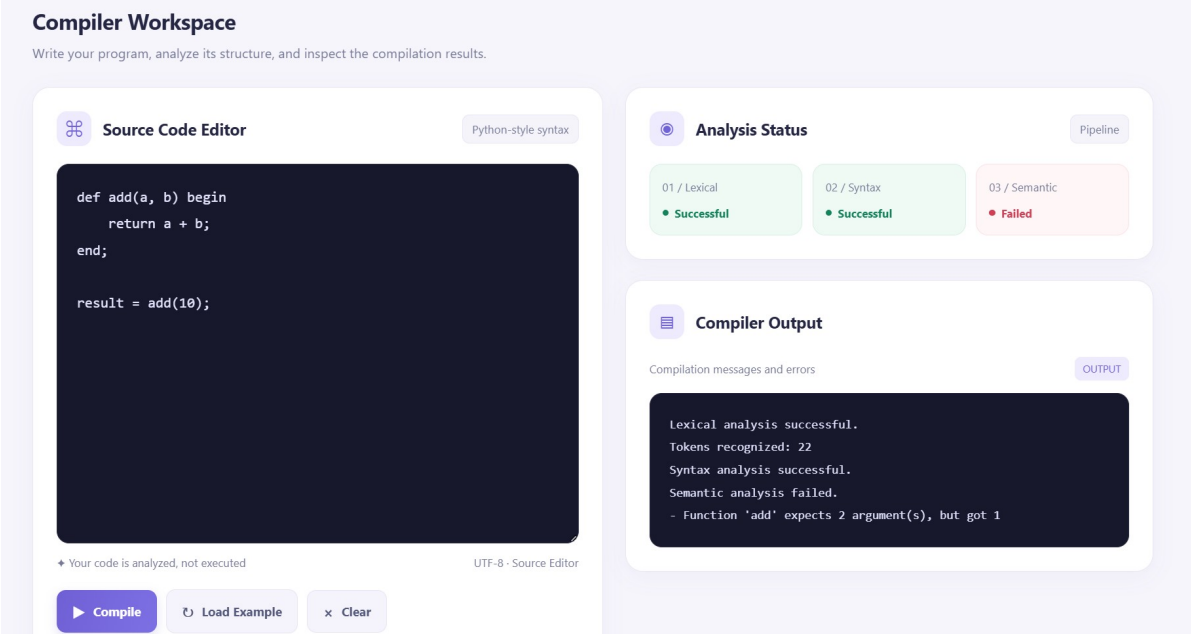


Figure 5: Semantic Analysis Failure

13. Individual Contribution Details

Student Name	Assigned Module / Role	Key Contributions & Outputs
Thomas Peter (Roll No: 64)	Grammar Design & Test Cases	Designed the CFG, identified terminals and the start symbol, and prepared valid and invalid test cases.
Zen Koshy (Roll No: 69)	Lexer & GitHub	Implemented lexer.py, tested token recognition, and managed repository setup and code commits.
Yash Bijo (Roll No: 68)	Parser & AST Construction	Worked on parser.py, grammar-based syntax validation, and AST construction using ast_nodes.py.
Ryan Jacob (Roll No: 40)	Semantic Checking, Documentation & Report Work	Worked on semantic_analyzer.py, documented the semantic rules and expected errors, and contributed to the project report.
Ruth Mariam Joseph (Roll No: 39)	Integration & Project Report	Integrated compiler modules with the application workflow, tested the combined system, and prepared the project report.
Teesa Thomas (Roll No: 61)	PPT & Documentation	Prepared the project presentation and contributed to project and technical documentation.

14. Conclusion

The Function Definition Compiler demonstrates important compiler front-end concepts through a Python-based implementation. The lexical analyzer identifies supported tokens, and the parser validates function definitions, parameter and argument lists, assignments, function calls, return statements, print statements, and arithmetic expressions. The AST node structures provide a representation of the program's syntax, while the semantic analyzer performs the semantic checks implemented in the project. The Git repository separates the grammar, lexer, parser, AST definitions, semantic analyzer, and application files, making the project easier to maintain and explain. Testing with valid and invalid programs helps demonstrate how the compiler responds to different inputs. Overall, the project provides a practical introduction to context-free grammars, lexical analysis, parsing, AST construction, semantic validation, and the organization of a compiler front end.